GENERALIZED QUINES

SONAT SÜER

October 28, 2026

Abstract. A quine is a program that prints its own source. A generalized quine is a program whose output, which may be an object of another kind, decodes back to the program. We give two hypotheses under which an explicit text is a generalized quine; the proof follows Ershov's construction of fixed points. The theorem is formalized in Lean 4, which computes the quine and proves it correct. We provide several examples.

1. Fixed points and quines

This section recalls how quines follow from Kleene's recursion theorem and records what the argument actually uses. It then turns that into a fixed point theorem for generalized quines.

1.1. **Quines and the recursion theorem.** A *quine* is a program that prints its own source text. Fix an acceptable numbering $(\varphi_n)_{n\in\mathbb{N}}$ of the partial computable functions. Kleene's second recursion theorem [2] states that for every total computable $f\colon \mathbb{N}\to\mathbb{N}$ there is an $e\in\mathbb{N}$ with $\varphi_e = \varphi_{f(e)}$.

Quines exist by a single application of the theorem. Indeed, if p is the total computable function given by the *s-m-n* theorem such that $\varphi_{p(n)}$ is the constant function with value n, then a fixed point e of p satisfies $\varphi_e = \varphi_{p(e)}$, so the program e outputs e.

1.2. **What the argument uses.** Two features of this argument suggest a generalization. First, the output e is compared with the program e only by equality. Nothing changes if programs output objects of some other kind, pictures for instance, provided that an object can be read back as a program. So codes and outputs need not live in the same set. Second, the argument does not need a Turing complete language. It uses that the program printing n can be computed from n. It also uses the two properties of the numbering that make the recursion theorem work.

Ershov isolated these properties [1]. A *numbering* of a set Y is a surjection $\gamma\colon\mathbb{N}\to Y$. It is *precomplete* if for every partial computable ψ there is a total computable g with $\gamma(g(n)) = \gamma(\psi(n))$ whenever $\psi(n)$ is defined.

Theorem 1.1 (Ershov [1])**.** *Let γ be a precomplete numbering and $f\colon\mathbb{N}\to\mathbb{N}$ a total computable function. Then there exists $n\in\mathbb{N}$ with $\gamma(f(n)) = \gamma(n)$.*

For the numbering $n\mapsto\varphi_n$ this is Kleene's theorem.

The fixed point in Ershov's proof is an explicit expression. Precompleteness provides a total computable function g with $\gamma(g(\langle c,x\rangle)) = \gamma(\varphi_c(x))$ whenever $\varphi_c(x)$ is defined. If a is an index of the total function $x\mapsto f(g(\langle x,x\rangle))$, then the fixed point is $n = g(\langle a,a\rangle)$. The hypotheses below ask only for what this expression needs, at the one point where it is used.

1.3. **Representation systems.** Throughout, a partial map $f\colon X\rightharpoonup Y$ is written as a map. An equation $f(x) = y$ asserts that $f(x)$ is defined and equal to y.

Definition 1.2. A *representation system* $\mathcal{S} = (T, P, \mathrm{disp}, \rho, \sigma)$ consists of sets T and P, a relation $\mathrm{disp}\subseteq T\times P$, a map $\rho\colon T\to P$ and a partial map $\sigma\colon P\rightharpoonup T$ such that

$$\sigma(\rho(t)) = t \quad \text{for all } t\in T.$$

The elements of T are *texts* and the elements of P are *representations*. We read $\mathrm{disp}(t,p)$ as "t displays as p", ρ as rendering and σ as scanning.

The relation disp need not be functional or total, so a text may display as nothing (it fails or does not terminate) or as several representations. Nothing about P or disp has to be computable.

Definition 1.3. Let $\mathcal{S}$ be a representation system. For $t \in T$ the *meaning* of t is the set

$$M(t) = \{\, s \in T : \text{there exists } p \in P \text{ with } \mathrm{disp}(t,p) \text{ and } \sigma(p) = s \,\}.$$

We say that t *means* s when $s \in M(t)$. A text e is a *generalized quine* if $e \in M(e)$.

Since $\sigma(\rho(e)) = e$, a generalized quine need not display as its own rendering. It is enough that it displays as something which the scanner reads as e.

Example 1.4. Let $T = P$ be the set of strings over an alphabet, with ρ and σ the identity. Let $\mathrm{disp}(t,o)$ hold when the program t terminates and prints o. Then $M(t)$ is the set of outputs of t. So a generalized quine is a quine in the usual sense.

1.4. **The hypotheses.** The first hypothesis says that rendering can be carried out inside the language. It plays the role of the computable map p above.

Definition 1.5. A *quotation* for $\mathcal{S}$ is a pair (D,q) of a subset $D \subseteq T$ and a map $q\colon T \to T$ such that for every $t \in D$ there exists $p \in P$ with

$$\mathrm{disp}(q(t),p) \quad \text{and} \quad \sigma(p) = \sigma(\rho(t)).$$

The text $q(t)$ is a program which draws t, or at least draws something that scans like the rendering of t. The set D records where this is possible. When representations have a bounded capacity, for instance, D contains only the texts that are short enough to be rendered.

Lemma 1.6. *If (D,q) is a quotation, then $t \in M(q(t))$ for every $t \in D$.*

Proof. For $t \in D$, the representation p of Definition 1.5 satisfies $\mathrm{disp}(q(t),p)$ and $\sigma(p) = \sigma(\rho(t)) = t$. □

The second hypothesis replaces the totalized universal function and the index a of Ershov's construction.

Definition 1.7. Let (D,q) be a quotation for $\mathcal{S}$. A *diagonal datum* for (D,q) is a triple (sub,a,O) of a map $\mathrm{sub}\colon T \times T \to T$, a text $a \in T$ and a subset $O \subseteq T$ such that

$$M\big(q(\mathrm{sub}(x,x))\big) \subseteq M\big(\mathrm{sub}(a,x)\big) \quad \text{for all } x \in O. \tag{1}$$

We call sub the *substitution*, a the *template* and O the *side condition*.

We think of $\mathrm{sub}(c,x)$ as the text that runs the code c on the data x. Condition (1) says that the template run on x means everything that the quotation of the diagonal $\mathrm{sub}(x,x)$ means. The set O restricts the data, typically to the texts that can be embedded in a program without escaping.

1.5. **The minimal fixed point theorem.**

Theorem 1.8. *Let (D,q) be a quotation for $\mathcal{S}$ and (sub,a,O) a diagonal datum for it. Put $e = \mathrm{sub}(a,a)$. If $a \in O$ and $e \in D$, then e is a generalized quine.*

Proof. Since $e \in D$, Lemma 1.6 gives $e \in M(q(e))$. Since $a \in O$, condition (1) with $x = a$ reads $M(q(e)) \subseteq M(e)$, because $\mathrm{sub}(a,a) = e$. Hence $e \in M(e)$. □

Remark 1.9. The fixed point $e = \mathrm{sub}(a,a)$ is Ershov's expression $g(\langle a,a\rangle)$. The substitution plays the role of $g(\langle\cdot,\cdot\rangle)$. The template plays the role of an index of $x \mapsto q(\mathrm{sub}(x,x))$. Condition (1) is used only at $x = a$ and only in one direction, so it can be verified for a single text in a language with no universal function at all. The hypothesis $e \in D$ is a genuine condition when representations have a finite capacity: the fixed point must itself be short enough to be rendered.

In the setting of Example 1.4, Theorem 1.8 produces a quine in the usual sense. In general it produces a text whose output, in whatever form the representations take, scans back to the text.

2. The theorem as a mechanism

Theorem 1.8 can be read as a machine. Its inputs are the hypotheses. Its output is an explicit text together with the guarantee that it is a generalized quine. This section lists the inputs and works through one example. The proofs that the example satisfies the hypotheses are in the formalization of Section 3.

2.1. **Inputs and output.** The inputs are the following.

(I1) A representation system $(T, P, \mathrm{disp}, \rho, \sigma)$: what texts are, what they display as, and how representations are read back.
(I2) A quotation (D, q): for each text $t \in D$, a program $q(t)$ that draws t.
(I3) A diagonal datum (sub, a, O): a way of running code on data, together with a template a which, run on data $x \in O$, behaves like the quotation of $\mathrm{sub}(x, x)$.
(I4) The two side conditions $a \in O$ and $\mathrm{sub}(a, a) \in D$.

The output is the text $e = \mathrm{sub}(a, a)$, a generalized quine.

2.2. **An example: a program that prints itself in braille.** Consider a language, such as Python, with string literals and a way to run a string as code. We want a program that prints its own source in the North American Computer Braille Code (NABCC), the 8-dot code of refreshable braille displays. Its output is not its own text. So it is a generalized quine but not a quine. The inputs are as follows.

Texts and representations: The set T consists of strings of characters. The set P consists of finite lists of braille cells, where the blank cell counts as a cell. A text displays as what the program prints. Rendering ρ replaces each character by its braille cell. Scanning σ reads the cells back.

Quotation: For a text t, the program $q(t)$ stores t as data and hands it to a fixed library L that prints text in braille. So $q(t)$ prints $\rho(t)$. The domain D is the texts in the characters that have cells.

Substitution: The program $\mathrm{sub}(c, x)$ stores the text x as data and then runs c as code, with access to the data. The side condition O is the texts that can be stored without escaping.

Template: The program a rebuilds the text $\mathrm{sub}(x, x)$ from its data x and hands it to L. So $\mathrm{sub}(a, x)$ prints $\rho(\mathrm{sub}(x, x))$, which is also what $q(\mathrm{sub}(x, x))$ prints.

The library avoids the characters that need escaping, so $a \in O$. The fixed point uses only characters that have cells, so $e \in D$. The output $e = \mathrm{sub}(a, a)$ is a program that stores the code a as data and runs it. Hence it prints $\rho(e)$, its own source in braille.

3. The Lean development

This section describes the formalization in general: the engine, how a language is plugged into it, what is proved, and what is trusted. The development uses Lean 4 [3] with its core library only, toolchain version 4.24.0. It lives in the directory `lean/` of the accompanying repository.

3.1. **The engine.** The module `QuineEngine.Engine` formalizes Section 1. A representation system is the structure `System T P`. Its field `disp` has type `T → P → Prop`, so representations never have to be computed. Its field `scan` takes values in `Option T`. The predicates `Means` and `IsQuine` are Definition 1.3. The hypotheses are structures whose fields are definitions and proofs. The structure `Quotation` is Definition 1.5, with fields `Dom`, `q` and `disp_q`. The structure `Diagonal` is Definition 1.7, with fields `sub`, `a`, `Ok` and `spec`. The lemma `Quotation.means_q` is Lemma 1.6.

The mechanism of Section 2 is the pair of an ordinary definition and a theorem:

```
Diagonal.quine D = D.sub D.a D.a,
Diagonal.isQuine D : D.Ok D.a → Q.Dom D.quine → S.IsQuine D.quine.
```

So an instance of the structures for a language outputs the text of a generalized quine, which Lean can evaluate. It also outputs a proof that the text is a generalized quine. The engine depends on no axiom.

3.2. **Instances.** A language is plugged in through a *file model*: a parser that recognizes the files of a fixed shape, an expression fragment with its evaluation, and the substitution, quotation and template written in that shape. Files of any other shape display as nothing, so the model is conservative. Its display relation states what the real language does on the files it accepts, typically that the file shows the rendering of the value of an expression handed to a library. Several instances share a file model, one for each of Python, SVG with JavaScript, PostScript, gnuplot, pdfLaTeX, OpenSCAD, G-code and Verilog.

The library of an instance, the code in the language that does the drawing, is read from a separate file by `include_str`, so it is an opaque text to the model. The concrete facts that an instance needs, such as $a \in O$, $e \in D$, or that the template parses to the expected expression, are closed by computation with `decide +kernel`.

The representations come in two kinds.

Concrete: Rendering and scanning are Lean functions, for instance a table of braille cells, Morse codes, serial frames or glyphs. Lean proves that scanning inverts rendering.

Abstract: Lean works with a structure `Codec`: a capacity C, a type of grids, an encoder and a decoder, with a proof that decoding inverts encoding on texts of length at most C. The theorem then holds for every codec of sufficient capacity, and no particular encoder is formalized.

In both kinds, a text outside the alphabet or above the capacity is rendered as a formal representation that no file displays as. This makes $\sigma \circ \rho$ the identity on all texts, while files only display representations that the real program can produce.

The executable `lake exe quine` evaluates every fixed point and writes it into the folder of its instance, together with, for the concrete representations, the representation that Lean predicts.

3.3. **What is proved.** For every instance, the development proves a theorem `quine_correct` of one of two forms. For a concrete representation it is

$$\texttt{system.IsQuine quine},$$

with no hypotheses. For an abstract representation it is

$$\forall\, Q : \texttt{Codec},\ C_0 \le Q.C \to \texttt{(system Q).IsQuine quine},$$

where C_0 is the capacity that the instance's library is meant to have. Each theorem is obtained from `Diagonal.isQuine` and depends only on the axioms `propext` and `Quot.sound`.

3.4. **What is trusted.** A proof about a model says nothing about the real program unless the model is faithful. The following are trusted.

(1) The Lean kernel and the compiled executable that writes the quines to disk.
(2) The file model: the real language behaves on files of the modelled shape as the display relation says.
(3) The library: it realizes the rendering, that is, it draws, prints or plays what ρ prescribes. For an abstract representation, the library implements a codec of capacity at least C_0.
(4) The physical display, when there is one: screen, printer, speaker, laser cutter or 3D printer.

3.5. **What is tested.** Each instance whose display relation is a model of a real language comes with a check, which runs the real tools outside the proof. The checks run the fixed point and, on random texts, the quotation, in the real language. They compare the output with the representation that Lean predicts, or decode it with independent decoders. Then they read it back to the text. These tests are evidence for the trusted items (2) and (3); they are not part of the proof.

4. The examples

This section describes the nineteen instances. For each it says what the quine does and gives the inputs of Section 2 and the form of the theorem, followed by notes. Throughout, L denotes the instance's library. The letter E denotes the expression of the template, which rebuilds $\mathrm{sub}(v, v)$ from the variable holding the data. Sizes are those of the generated files.

4.1. **Underload.** Underload is a small stack language whose programs are strings of one-character commands acting on a stack of strings. The quine is the ordinary quine `(:aSS):aSS`.

Representation: Texts and representations are strings. A program displays as what it prints. Rendering and scanning are identities, as in Example 1.4.

Inputs: The quotation is $q(t) =$ `(`t`)S`. The substitution is $\mathrm{sub}(c, x) =$ `(`x`)`c. The template is $a =$ `:aSS`. Both the side condition O and the domain D are the strings with balanced parentheses.

Theorem: Unconditional.

Notes: The display relation is what an interpreter for the language, written in Lean with fuel, prints. So, unlike most of the examples, nothing about the display is trusted.

4.2. **An SVG file that draws a QR code of itself.** The file `qr-code.svg` (1,822 bytes) runs a script that draws a QR code whose payload is the file.

Representation: A QR code, version 31, level L, byte mode, through an abstract codec.

Inputs: Files have the shape

`<svg ...><script>v="x";eval(arg)</script></svg>`.

The quotation $q(t)$ binds t with `\xHH` escapes and evaluates L`;drawQR(v)`. The substitution $\mathrm{sub}(c, x)$ binds x and evaluates c, or `v` when $c = x$. The template a is L`;drawQR(E)`. The side condition O is the texts that are safe to embed raw. The domain D is the Latin-1 texts within the capacity.

Theorem: Relative to a codec of capacity at least 1,840.

Notes: The script contains no `<`, `&` or `]]>`, so the XML parser passes it through unchanged, which makes the model faithful.

4.3. **A formula that plots itself.** A two-line text `0<E,k=` over the decimal value of k denotes the set of points $(x, y) \in \mathbb{N}^2$ with $0 < E(x, y, k)$, where E is an arithmetic term. The quine (807 bytes) is a formula whose solution set is exactly the typesetting of its own text, constant included.

Representation: Subsets of $\mathbb{N}^2$. Rendering typesets the two lines with a 3×5 font, right-aligned; scanning reads the glyphs back.

Inputs: The quotation $q(t)$ is a formula whose constant interleaves the lines of t. The substitution $\mathrm{sub}(c, x)$ is the formula with first line c and constant the number written by x. The template a is the first line `0<E,k=`. The side condition O requires both lines of $\mathrm{sub}(x, x)$ to fit. The domain D is the two-line texts that are well formed.

Theorem: Unconditional.

Notes: The idea goes back to Tupper's self-referential formula [5], whose plot shows the formula itself. Tupper's formula draws itself only in a strip, and its constant is supplied from outside. Here the constant is part of the formula and appears in the picture. The formula is plotted over the whole quadrant. The display relation is the solution set, which Lean defines from the formula itself. So, unlike most of the examples, nothing about the display is trusted.

4.4. **A PostScript file that prints a QR code of itself.** The file prints a page with a QR code of itself; it is `qr-code.ps`, of 2,820 bytes.

Representation: A QR code of version 40, level L, byte mode and mask 0, through an abstract codec.

Inputs: Files are `%!PS` followed by one of

`(x)dup cvx exec`, `(x)(c)cvx exec`, `<hex>(c)cvx exec`.

The quotation $q(t)$ pushes t in hexadecimal and runs L `Q showpage`. The substitution $\mathrm{sub}(c, x)$ pushes x and runs c, or `dup cvx exec` when $c = x$. The template a is L followed by a tail of hexadecimal strings, `exch` and `cat` that rebuilds the file, then `Q showpage`. The side condition O is printable ASCII without backslash and with balanced parentheses. The domain D is the Latin-1 texts within the capacity.

Theorem: Relative to a codec of capacity at least 2,953.

Notes: The code sits in a `(...)` string, which is why it must have balanced parentheses and no backslash.

4.5. **A gnuplot script that plots a QR code of itself.** The script `qr-code.gp` (2,214 bytes) plots a QR code of itself.

Representation: A QR code, version 40, level L, byte mode, mask 0, through an abstract codec.

Inputs: Files have the shape `v=LIT;eval ARG`. The quotation $q(t)$ binds t in octal escapes and evaluates `s=v;`L. The substitution $\mathrm{sub}(c, x)$ binds x and evaluates c, or `v` when $c = x$. The template a is `s=E;`L. The side condition O is printable ASCII without `'`. The domain D is the printable texts within the capacity.

Theorem: Relative to a codec of capacity at least 2,953.

Notes: The encoder is written in gnuplot. The code sits in a `'...'` literal, so it writes `'` as `sprintf("%c",39)`. It is one line long because gnuplot literals cannot span lines. Gnuplot has no character-code function, so a character's code is its position in a string of the printable characters. Routines that write arrays are strings run with `eval`.

4.6. **A pdfLaTeX file that typesets a color QR code of itself.** The file `qr-code.tex` (6,430 bytes) typesets a color QR code of itself as a grid of colored cells, each carrying a Greek letter or an operator.

Representation: Three QR codes, version 40, level L, one in each RGB channel, through an abstract codec of three times the capacity. Scanning separates the channels, decodes each and concatenates.

Inputs: Files have the shape $PRE\,x$`|`, where PRE reads every byte up to `|` verbatim and runs the captured code x. The template is `\iffalse` PRE `\fi`, then L, then a part that rebuilds the file, splits it into thirds and encodes them. The quotation is a second, octal-encoded code that outputs a fixed text. The side condition O is the printable texts without `|` that hold a copy of PRE. The domain D is the printable texts within the capacity.

Theorem: Relative to a codec of capacity at least 8,859.

Notes: The template reads PRE back from its `\iffalse` copy instead of rebuilding it from character codes. The expl3 encoder is golfed from about 8 KB to about 5 KB; without these two measures the file needed four QR codes. The page uses about 4.2 of the 5 million words of pdfTeX's main memory, which is why the cells avoid math mode and boxes. The kernel computations are split along the file's own concatenations, so that each piece reduces without exhausting the kernel's stack.

4.7. **An SVG file that draws its own source.** The file `text.svg` (1,178 bytes) draws its own source as pixels.

Representation: Bitmaps. Rendering typesets the text with the 5×7 X11 misc-fixed font, 80 characters a row; scanning reads every cell back with the same font.

Inputs: The SVG file model, with L`;drawText(E)` as the template. The side condition O is the texts that are safe to embed raw. The domain D is the printable texts that do not end in a space.

Theorem: Unconditional: Lean proves that scanning inverts typesetting, so no decoder is trusted.

4.8. **An SVG file that draws a color QR code of itself.** The file `color-qr.svg` (2,045 bytes) draws a color QR code of itself.

Representation: Three QR codes, version 31, level L, overlaid in the red, green and blue channels, through an abstract codec.

Inputs: The SVG file model, with L`;drawColorQR(E)` as the template.

Theorem: Relative to a codec of capacity at least 5,520, three times 1,840.

4.9. **A PostScript file that prints a color QR code of itself.** The file `color-qr.ps` (3,192 bytes) prints the color QR code of the previous example.

Representation: As for the SVG color QR code.

Inputs: The PostScript file model, with the template of the PostScript QR code.

Theorem: Relative to a codec of capacity at least 5,520.

Notes: Each module is 3 points, so the symbol fits on A4 and Letter paper.

4.10. **An SVG file that draws an Aztec code of itself.** The file `aztec.svg` (2,072 bytes) draws a full-range Aztec code, 32 layers and 151 × 151 modules, of itself.

Representation: An Aztec code, through an abstract codec.

Inputs: The SVG file model, with L`;drawAztec(E)` as the template.

Theorem: Relative to a codec of capacity at least 2,278, the most bytes that fit in 32 layers with at least 3 error-correction codewords whatever the bit stuffing.

Notes: The encoder avoids `<`, `&`, `"` and backslash, so shifts become divisions by powers of two and masks become `%`.

4.11. **A PostScript file that prints an Aztec code of itself.** The file `aztec.ps` (2,026 bytes) prints the Aztec code of the previous example.

Representation: As for the SVG Aztec code.

Inputs: The PostScript file model, with the template of the PostScript QR code.

Theorem: Relative to a codec of capacity at least 2,278.

Notes: The readable encoder is 3,165 bytes, more than the code can hold. So the library is a loader of 1,936 bytes, which decodes a Base64 string, decompresses it with PostScript's `FlateDecode` filter and executes the result. To the model it is an opaque text like any other library.

4.12. **A Python program that prints itself in braille.** The program `braille.py` (1,191 bytes) prints its own source in NABCC, as Unicode braille cells. It is the example of Section 2.2.

Representation: Strings of NABCC cells. Newlines stay newlines.

Inputs: Files have one of the shapes

`v="""x""";exec(v)` or `v="""x""";exec("""c""")`,

which bind the string x to `v` and execute x or c. The quotation $q(t)$ binds t in hexadecimal and executes L`braille(v)`. The substitution $\mathrm{sub}(c, x)$ binds x and executes c, or `v` when $c = x$. The template a is L`braille(E)`, where E is the expression

`'v='+chr(34)+chr(34)+chr(34)+v+chr(34)+chr(34)+chr(34)+';exec(v)'+chr(10)`.

The side condition O is the texts that can sit inside `"""..."""`: printable ASCII and newlines, without `"` or backslash. The domain D is printable ASCII and newlines.

Theorem: Unconditional.

Notes: The Python file model is shared by the next four examples. Only the library, the function it defines and the alphabet change.

4.13. **A Python program that blinks itself in Morse code.** The program `morse.py` (1,339 bytes) opens a window and blinks its own source in International Morse code, white for light and black for dark, for about 17 minutes.

Representation: Sequences of Morse codes, one per character, with the prosign AA for a line break and a longer gap for a space.

Inputs: The Python file model, with L`blink(E)` as the template. The domain D is the texts in the 57 characters that have codes.

Theorem: Unconditional.

Notes: Morse code has no case and few symbols, so the program is written in lower-case letters, digits, a few punctuation marks, space and newline.

4.14. **A Python program that plays itself as a modem.** The program `modem.py` (1,154 bytes) plays its own source as a 300-baud Bell 103 modem, the classic dial-up sound, for about 40 seconds.

Representation: Sequences of 8N1 serial frames, one per byte, each bit a tone of 1,270 Hz (mark) or 1,070 Hz (space).

Inputs: The Python file model, with L`play(E)` as the template. The domain D is the ASCII texts.

Theorem: Unconditional.

Notes: The sound is played with the pip package `miniaudio`, whose wheels include the audio library, so the program needs no system audio package.

4.15. **A Python program that hides itself in autostereograms.** The program `stereogram.py` (3,345 bytes) writes a PDF of 140 random-dot autostereograms [4] in which its own source is hidden as raised 16-segment characters, three rows of eight to a page, after a title page.

Representation: Sequences of 16-segment patterns, one per character, from a table of 62 characters; a line break is a filled cell.

Inputs: The Python file model, with L`draw(E)` as the template. The domain D is the texts in the 62 characters of the table.

Theorem: Unconditional.

Notes: Sixteen-segment characters have no case, so the program has no capital letters; it writes the capitals that PDF needs with a small helper. A 5×7 font and QR codes proved illegible in stereograms. Segment characters read well once each was jittered off the grid. The PDF is written with `zlib` alone and stays under 50 MB.

4.16. **A Python program whose sound shows its own QR code.** The program `spectrogram.py` (2,918 bytes) makes a 37-second sound whose spectrogram is a QR code of the program. It plays the sound, or writes it as a WAV file.

Representation: A QR code, version 40, level L, through an abstract codec. In the spectrogram each module column is 0.2 s. Each module row is a pair of tones 10 Hz apart, the rows being 50 Hz apart. A light module sounds, while a dark one is silent.

Inputs: The Python file model, with L`play(E)` as the template. The domain D is the ASCII texts within the capacity.

Theorem: Relative to a codec of capacity at least 2,953.

Notes: The QR encoder and the synthesis are pure Python. The library is compacted so that the program fits in version 40. The sound design was chosen by scanning test spectrograms with a phone. One tone per row gave stripes, while tones filling the row blurred into the dark rows.

4.17. **A Verilog circuit that sends its own description on a serial pin.** The file `quine.v` (3,521 bytes) is a synthesizable circuit that, when simulated, drives its `tx` pin with 9600-baud serial carrying the file.

Representation: Sequences of 8N1 frames, shared with the modem.

Inputs: Files are a line `‘define s "x"` followed by code. Verilog has no eval, so this is a data and code quine. The quotation $q(t)$ defines `s` as t, escaped, followed by the circuit with `full = 0`, which sends `s`. The substitution $\mathrm{sub}(c, x)$ defines `s` as x, followed by the code c. The template a is the circuit with `full = 1`, which sends the definition line and then `s`. The side condition O is printable ASCII and newlines. The domain D is the nonempty such texts.

Theorem: Unconditional.

Notes: Icarus Verilog's preprocessor removes comments inside a `‘define`, even inside a string literal, so the code has no comments.

4.18. **An OpenSCAD model of itself in braille.** The file `braille.scad` (1,838 bytes) is a model of braille plates, in tactile dimensions, that emboss the file's own source in 6-dot Braille ASCII.

Representation: Sequences of Braille ASCII cells; a line break is a blank row.

Inputs: Files are `s="x";` followed by code. OpenSCAD has no eval, so the substitution $\mathrm{sub}(c, x)$ binds x to `s` and runs the code c. The template a is L`braille(E);`. The side condition O is printable ASCII without `"` or backslash. The domain D is the texts in the 64 characters of Braille ASCII and newline.

Theorem: Unconditional.

Notes: Braille ASCII has 64 cells and no case, so the program avoids capitals, `{ | } ~` and the backtick. Its module body is a single statement. Top-level shapes are unioned implicitly. The function `pow` replaces `^`. The code is one line, because OpenSCAD drops raw newlines inside a string literal. It writes `"` with `chr`.

4.19. **A G-code program that laser-cuts a stencil of itself.** The program `stencil.ngc` (6,624 bytes), run on a laser cutter, cuts its own source out of a sheet as stencil lettering of 5×7 glyphs, one bar per run of pixels, so that the strips between rows hold every letter together.

Representation: Sequences of glyphs of the font of the SVG text example.

Inputs: G-code has no strings, so a file is a data block that stores the code in numbered parameters, followed by code. The quotation $q(t)$ stores t and runs L `O100CALL[0]`. The substitution $\mathrm{sub}(c, x)$ stores x and runs c. The template a is L `O100CALL[1]`, which cuts the data block and then the data. Both the side condition O and the domain D are the texts in the program's alphabet that fit the data block.

Theorem: Unconditional.

Notes: The program uses 40 characters: newline, digits, a few symbols and the capitals it needs. It has no spaces, because LinuxCNC ignores them.

References

[1] Yu. L. Ershov, *Theorie der Numerierungen I*, Z. Math. Logik Grundlagen Math. **19** (1973), 289–388.

[2] S. C. Kleene, *On notation for ordinal numbers*, J. Symbolic Logic **3** (1938), 150–155.

[3] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, in: Automated Deduction – CADE 28, Lecture Notes in Comput. Sci. **12699**, Springer, Cham, 2021, 625–635.

[4] H. W. Thimbleby, S. Inglis and I. H. Witten, *Displaying 3D images: algorithms for single-image random-dot stereograms*, Computer **27** (1994), no. 10, 38–48.

[5] J. Tupper, *Reliable two-dimensional graphing methods for mathematical formulae with two free variables*, in: Proceedings of SIGGRAPH 2001, ACM, New York, 2001, 77–86.